



5.1. Data Grouping and Aggregation

Previous Section:

 [4.2. Inserting, Deleting, and Updating Data Values](#)

Contents:

[1. Basic Aggregation](#)

[2. Advanced Aggregation](#)

[3. Aggregation and Sorting](#)

[4. Aggregation with Lambda Functions](#)

[Summary](#)

[References](#)

Pandas allows you to **group similar records together** and calculate summary statistics for each group. This process, known as **aggregation**, is one of the most powerful features in data analysis because it transforms detailed records into meaningful summaries.

For example, instead of viewing every student's GPA individually, you can quickly calculate the **average GPA for each major**, **count the number of students in each department**, or **find the highest GPA in every group**.

Throughout this section, we will continue using the following dataset.

```
import numpy as np
import pandas as pd

# Reproducible randomness
np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]
```

```
n = 20

df = pd.DataFrame({
    "Name": np.array(names),
    "Age": np.random.randint(18, 21, n),
    "Major": np.random.choice(majors, n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

print(df.head(10))
```

1. Basic Aggregation

The `groupby()` method divides the dataframe into groups based on one or more columns. After grouping, you can apply aggregation functions such as **mean**, **sum**, **count**, **minimum**, **maximum**, and **median**.

Some of the most common aggregation methods include:

- `mean()` – Average value
- `median()` – Middle value
- `sum()` – Sum of all values
- `count()` – Number of non-missing values
- `min()` – Smallest value
- `max()` – Largest value

For example, suppose we want to calculate the average GPA for each major.

```
df.groupby("Major")["GPA"].mean()
```

Major	GPA
AI	3.12
CSC	3.0625
IT	2.89333333
SW	2.74625

We can also compute multiple statistics simultaneously.

```
df.groupby("Major")["GPA"].agg(["mean", "median", "min", "max", "count"])
```

Major	mean	median	min	max	count
-------	------	--------	-----	-----	-------

AI	3.12	3.03	2.34	3.93	5
CSC	3.0625	3.115	2.2	3.82	4
IT	2.893333333	3.22	2.09	3.37	3
SW	2.74625	2.895	2.07	3.57	8

2. Advanced Aggregation

The `groupby()` method also supports grouping by **multiple columns**. In addition, the `agg()` method allows different aggregation functions to be applied to different columns.

For example, we can group students by both **Major** and **Age**.

```
df.groupby(["Major", "Age"]).mean(numeric_only=True)
```

Major	Age	GPA
AI	19	2.34
AI	20	3.315
CSC	18	2.61
CSC	19	3.82
CSC	20	2.91
IT	18	2.09
IT	19	3.37
IT	20	3.22
SW	18	3.21
SW	19	2.433333333
SW	20	2.52



By default, Pandas returns the aggregated columns as a multi-index. You can make them separate columns by using the `unstack()` method

```
# You can also make them separate columns instead of a multi-index by using the unstack() method
df.groupby(["Major", "Age"]).mean(numeric_only=True).unstack()
```

	GPA	GPA	GPA
Age	18	19	20
Major			
AI	NaN	2.34	3.315
CSC	2.61	3.82	2.91
IT	2.09	3.37	3.22

SW	3.21	2.43333333	2.52
----	------	------------	------

We can also specify different aggregation functions for different columns.

```
df.groupby("Major").agg({
    "Age": "mean",
    "GPA": ["mean", "max", "min", "count"]
})
```

the `numeric_only=True` parameter ensures that only numeric columns are included in the aggregation

	Age	GPA	GPA	GPA	GPA
	mean	mean	max	min	count
Major					
AI	19.8	3.12	3.93	2.34	5
CSC	19.25	3.0625	3.82	2.2	4
IT	19	2.89333333	3.37	2.09	3
SW	18.875	2.74625	3.57	2.07	8

This produces a summary where each column is aggregated using the most appropriate statistical measure.

3. Aggregation and Sorting

After aggregation, it is common to sort the results to identify the highest or lowest values.

For example, suppose we want to rank majors by their average GPA.

```
df.groupby("Major")["GPA"].mean().sort_values(ascending=True)
```

Major	GPA
SW	2.74625
IT	2.89333333
CSC	3.0625
AI	3.12

Similarly, we can sort using multiple aggregated columns.

```
summary = df.groupby("Major").agg({
    "Age": "mean",
```

```

        "GPA": "mean"
    })

summary.sort_values(
    by=["GPA", "Age"],
    ascending=[False, True]
)

```

Sorting aggregated results makes it much easier to compare groups and identify trends.

Major	Age	GPA
AI	19.8	3.12
CSC	19.25	3.0625
IT	19.0	2.89333333
SW	18.875	2.74625

4. Aggregation with Lambda Functions

The `agg()` method also accepts **lambda functions**, allowing you to define custom aggregation operations that are not provided by Pandas.

For example, suppose we want to calculate the GPA range (maximum – minimum) for each major.

```
df.groupby("Major")["GPA"].agg(lambda x: x.max() - x.min())
```

Major	GPA
AI	1.59
CSC	1.62
IT	1.28
SW	1.5

We can also combine built-in functions with lambda functions.

```
df.groupby("Major")["GPA"].agg([
    "mean",
    "max",
    "min",
    ("GPA Range", lambda x: x.max() - x.min())
])
```

Major	mean	max	min	GPA Range
AI	3.12	3.93	2.34	1.59

CSC	3.0625	3.82	2.2	1.62
IT	2.893333333	3.37	2.09	1.28
SW	2.74625	3.57	2.07	1.5

We will discuss lambda functions in more detail in a later chapter. For now, simply think of them as short, anonymous functions that allow you to perform custom calculations during aggregation.

Summary

The `groupby()` method is one of the most important tools in Pandas because it allows you to summarize large datasets efficiently.

By combining grouping with aggregation functions, sorting, and custom lambda expressions, you can quickly generate meaningful insights from raw data.

These techniques are widely used in data analytics for reporting, exploratory analysis, and preparing data for visualization or machine learning.

To practice further, complete Lab 05 of the Data Analytics Hands-On Lab.

 [Lab 07: Data Analytics with Python](#)

References

<https://pandas.pydata.org/pandas-docs/version/2.2/reference/api/pandas.core.groupby.DataFrameGroupBy.aggregate.html>

<https://www.geeksforgeeks.org/pandas/pandas-groupby/>

Next Section:

 [5.2. Data Grouping and Transforming](#)